

Spring REST using Spring Boot 3 (Week – 3)

Hands on 1

Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as “com.cognizant”
3. Change Artifact Id as “spring-learn”
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using ‘mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456’ command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

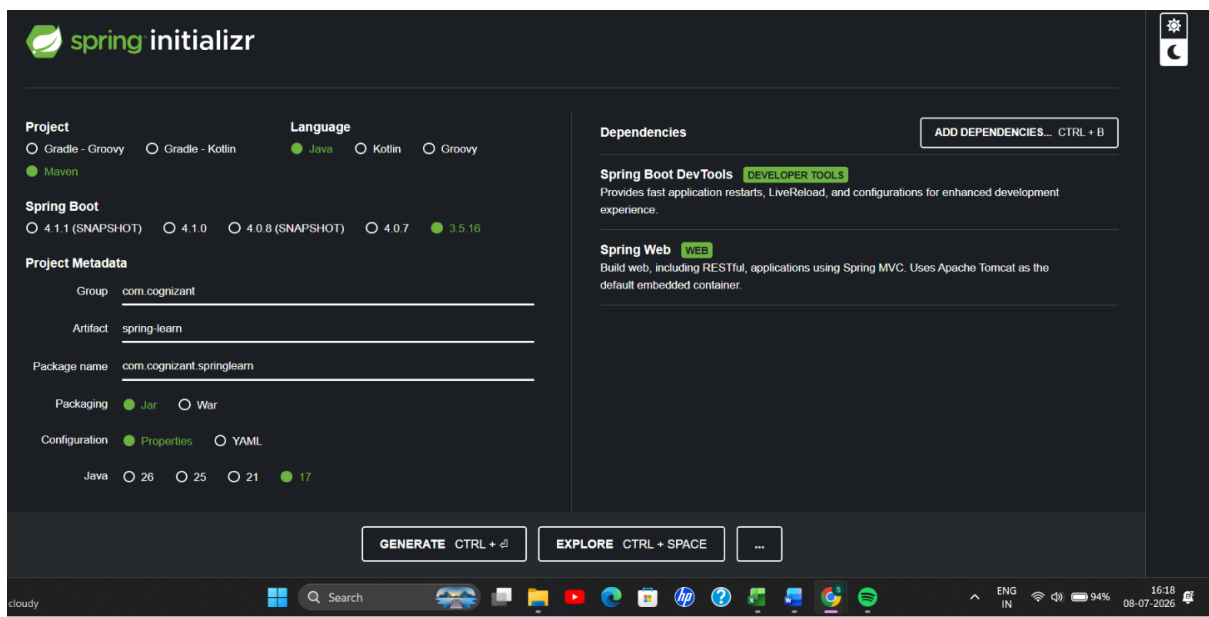
SME to walk through the following aspects related to the project created:

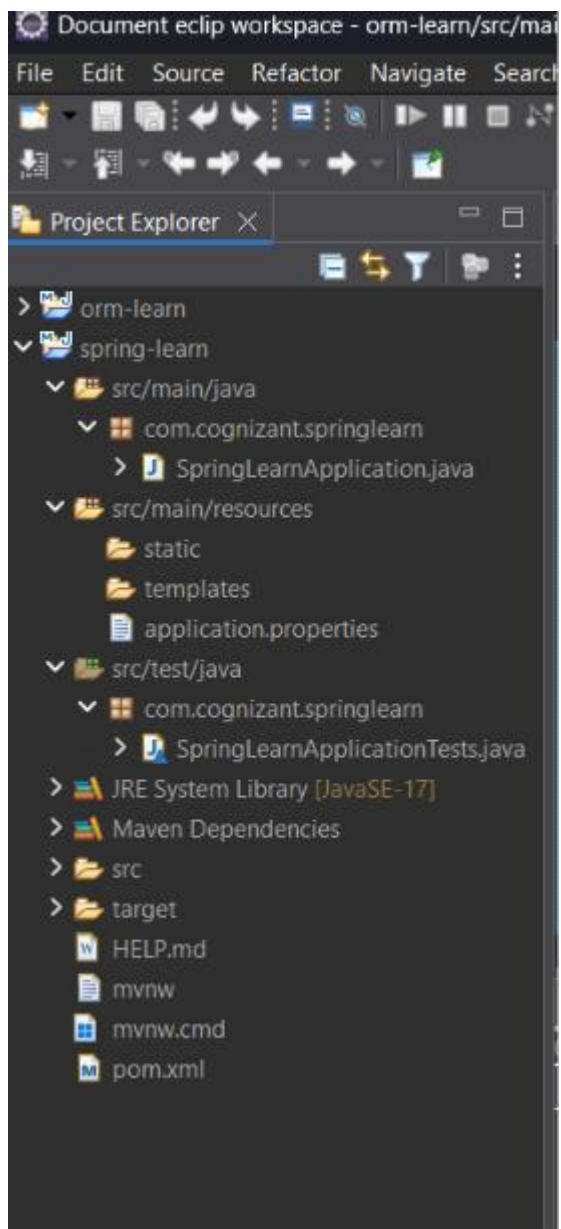
1. src/main/java - Folder with application code
2. src/main/resources - Folder for application configuration
3. src/test/java - Folder with code for testing the application
4. SpringLearnApplication.java - Walkthrough the main() method.
5. Purpose of @SpringBootApplication annotation

6. pom.xml

1. Walkthrough all the configuration defined in XML file
2. Open 'Dependency Hierarchy' and show the dependency tree.

OUTPUT:





```
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7
8 @SpringBootApplication
9 public class SpringLearnApplication {
10
11     private static final Logger LOGGER =
12         LoggerFactory.getLogger(SpringLearnApplication.class);
13
14     public static void main(String[] args) {
15
16         SpringApplication.run(SpringLearnApplication.class, args);
17
18         LOGGER.info("Inside main");
19     }
20 }
```

SpringLearnApplication [Java Application] C:\Users\shiva\Downloads\eclipse-je-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\java.exe (08-Jul-2026, 4:58:46 pm elapsed 0)

Spring Boot (v3.5.16)

```
2026-07-08T16:58:48.236+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.11 with PID 165
2026-07-08T16:58:48.239+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "def
2026-07-08T16:58:48.335+05:30 INFO 16592 --- [spring-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-pr
2026-07-08T16:58:48.335+05:30 INFO 16592 --- [spring-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'login
2026-07-08T16:58:49.613+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-08T16:58:49.630+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-08T16:58:49.630+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
```

SpringLearnApplication [Java Application] C:\Users\shiva\Downloads\eclipse-je-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\java.exe (08-Jul-2026, 4:58:46 pm elapsed 002:01) [pid: 16592]

Spring Boot (v3.5.16)

```
2026-07-08T16:58:48.236+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.11 with PID 16592 (C:\Users\shiva\Down
2026-07-08T16:58:48.239+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-08T16:58:48.335+05:30 INFO 16592 --- [spring-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to
2026-07-08T16:58:48.335+05:30 INFO 16592 --- [spring-learn] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property
2026-07-08T16:58:49.613+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-08T16:58:49.630+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-08T16:58:49.630+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-07-08T16:58:49.670+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-08T16:58:49.671+05:30 INFO 16592 --- [spring-learn] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1335 ms
2026-07-08T16:58:49.237+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : LiveReload server is running on port 35729
2026-07-08T16:58:50.305+05:30 INFO 16592 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-07-08T16:58:50.320+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 2.768 seconds (process running for 3.529)
2026-07-08T16:58:50.327+05:30 INFO 16592 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Inside main
```

Dependency Hierarchy [test]

Filter:

Dependency Hierarchy

- spring-boot-starter-web : 3.5.16 [compile]
 - spring-boot-starter : 3.5.16 [compile]
 - spring-boot : 3.5.16 [compile]
 - spring-boot-autoconfigure : 3.5.16 [compile]
 - spring-boot-starter-logging : 3.5.16 [compile]
 - logback-classic : 1.5.34 [compile]
 - logback-core : 1.5.34 [compile]
 - slf4j-api : 2.0.18 (managed from 2.0.17) [compile]
 - log4j-to-slf4j : 2.24.3 [compile]
 - log4j-api : 2.24.3 [compile]
 - slf4j-api : 2.0.18 (managed from 2.0.16) [compile]
 - jul-to-slf4j : 2.0.18 [compile]
 - slf4j-api : 2.0.18 [compile]
 - jakarta.annotation-api : 2.1.1 [compile]
 - spring-core : 6.2.19 [compile]
 - snakeyaml : 2.4 [compile]
- spring-boot-starter-json : 3.5.16 [compile]
 - spring-boot-starter : 3.5.16 [compile]
 - spring-web : 6.2.19 [compile]
- jackson-databind : 2.21.4 [compile]
 - jackson-annotations : 2.21 [compile]
 - jackson-core : 2.21.4 [compile]
- jackson-datatype-jdk8 : 2.21.4 [compile]
 - jackson-core : 2.21.4 [compile]
 - jackson-databind : 2.21.4 [compile]
- jackson-datatype-jsr310 : 2.21.4 [compile]
 - jackson-annotations : 2.21 [compile]
 - jackson-core : 2.21.4 [compile]

Resolved Dependencies

- accessors-smart : 2.5.2 [test]
- android-json : 0.0.20131108.vaadin1 [test]
- apiguardian-api : 1.1.2 [test]
- asm : 9.7.1 [test]
- assertj-core : 3.27.7 [test]
- awaitility : 4.2.2 [test]
- byte-buddy : 1.17.8 [test]
- byte-buddy-agent : 1.17.8 [test]
- hamcrest : 3.0 [test]
- jackson-annotations : 2.21 [compile]
- jackson-core : 2.21.4 [compile]
- jackson-databind : 2.21.4 [compile]
- jackson-datatype-jdk8 : 2.21.4 [compile]
- jackson-datatype-jsr310 : 2.21.4 [compile]
- jackson-module-parameter-names : 2.21.4 [compile]
- jakarta.activation-api : 2.1.4 [test]
- jakarta.annotation-api : 2.1.1 [compile]
- jakarta.xml.bind-api : 4.0.5 [test]
- jsonassert : 1.5.3 [test]
- json-path : 2.9.0 [test]
- json-smart : 2.5.2 [test]
- jul-to-slf4j : 2.0.18 [compile]
- junit-jupiter : 5.12.2 [test]
- junit-jupiter-api : 5.12.2 [test]
- junit-jupiter-engine : 5.12.2 [test]
- junit-jupiter-params : 5.12.2 [test]
- junit-platform-commons : 1.12.2 [test]

Hands on 4

Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

Code	Name
US	United States
DE	Germany
IN	India
JP	Japan

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named country.xml.
- Create a bean tag in spring configuration for country and set the property and values

```
<bean id="country" class="com.cognizant.springlearn.Country">  
  <property name="code" value="IN" />  
  <property name="name" value="India" />  
</bean>
```

- Create Country class with following aspects:
 - Instance variables for code and name
 - Implement empty parameter constructor with inclusion of debug log within the constructor with log message as “Inside Country Constructor.”

- Generate getters and setters with inclusion of debug with relevant message within each setter and getter method.
- Generate toString() method
- Create a method displayCountry() in SpringLearnApplication.java, which will read the country bean from spring configuration file and display the country details. ClassPathXmlApplicationContext, ApplicationContext and context.getBean("beanId", Country.class). Refer sample code for displayCountry() method below.

```
ApplicationContext context = new
ClassPathXmlApplicationContext("country.xml");

Country country = (Country) context.getBean("country", Country.class);

LOGGER.debug("Country : {}", country.toString());
```

- Invoke displayCountry() method in main() method of SpringLearnApplication.java.
- Execute main() method and check the logs to find out which constructors and methods were invoked.

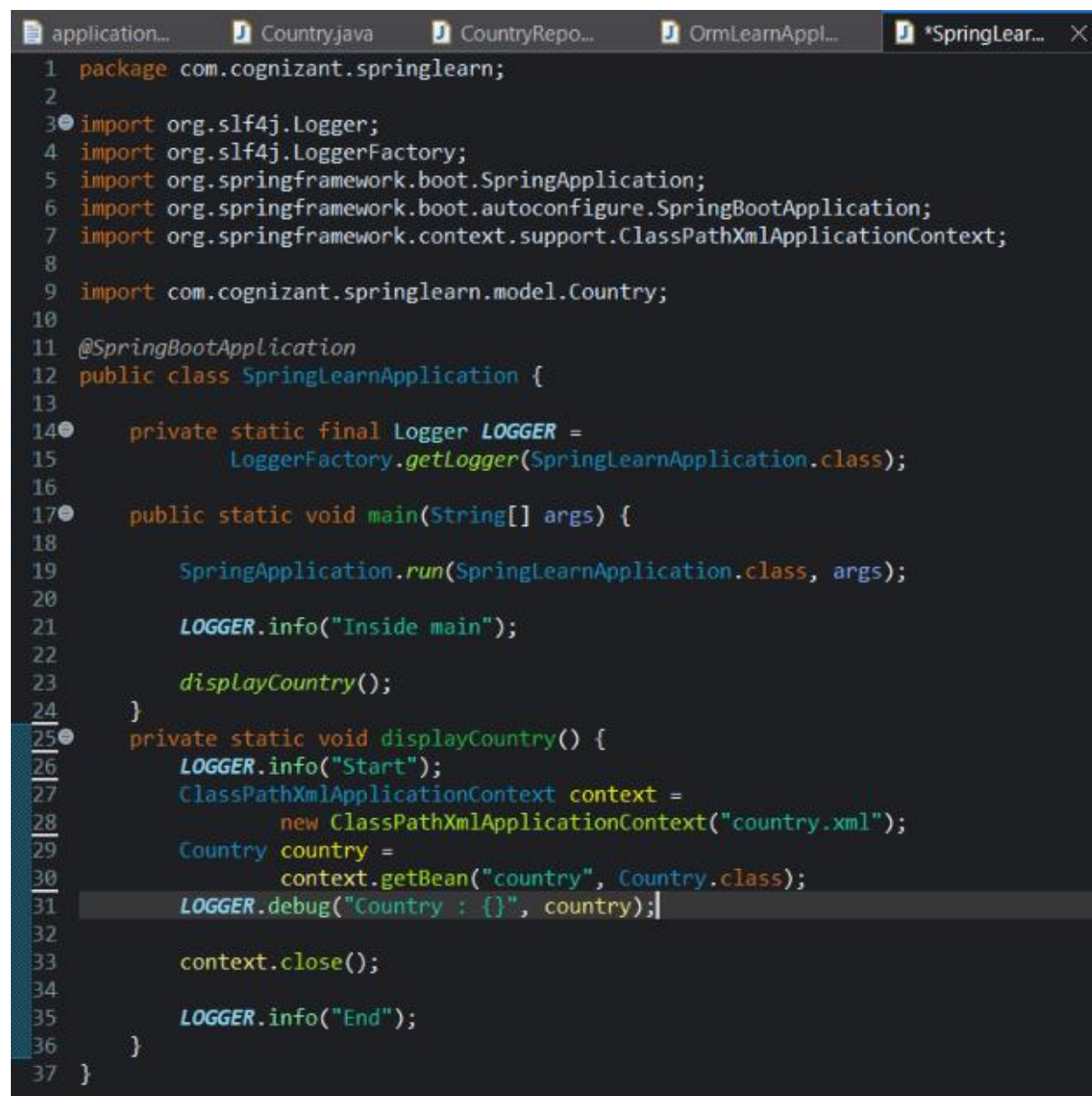
SME to provide more detailing about the following aspects:

- bean tag, id attribute, class attribute, property tag, name attribute, value attribute
- ApplicationContext, ClassPathXmlApplicationContext

What exactly happens when context.getBean() is invoked

OUTPUT:

SpringLearnApplication.java



```
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7 import org.springframework.context.support.ClassPathXmlApplicationContext;
8
9 import com.cognizant.springlearn.model.Country;
10
11 @SpringBootApplication
12 public class SpringLearnApplication {
13
14     private static final Logger LOGGER =
15         LoggerFactory.getLogger(SpringLearnApplication.class);
16
17     public static void main(String[] args) {
18
19         SpringApplication.run(SpringLearnApplication.class, args);
20
21         LOGGER.info("Inside main");
22
23         displayCountry();
24     }
25     private static void displayCountry() {
26         LOGGER.info("Start");
27         ClassPathXmlApplicationContext context =
28             new ClassPathXmlApplicationContext("country.xml");
29         Country country =
30             context.getBean("country", Country.class);
31         LOGGER.debug("Country : {}", country);
32
33         context.close();
34
35         LOGGER.info("End");
36     }
37 }
```

Country.java

```
1 package com.cognizant.springlearn.model;
2 import org.slf4j.Logger;
3
4 public class Country {
5     private static final Logger LOGGER =
6         LoggerFactory.getLogger(Country.class);
7
8     private String code;
9     private String name;
10    public Country() {
11        LOGGER.debug("Inside Country Constructor.");
12    }
13    public String getCode() {
14        LOGGER.debug("Inside getCode()");
15        return code;
16    }
17    public void setCode(String code) {
18        LOGGER.debug("Inside setCode()");
19        this.code = code;
20    }
21    public String getName() {
22        LOGGER.debug("Inside getName()");
23        return name;
24    }
25    public void setName(String name) {
26        LOGGER.debug("Inside setName()");
27        this.name = name;
28    }
29    @Override
30    public String toString() {
31        return "Country [code=" + code +
32            ", name=" + name + "]";
33    }
34 }
```

country.xml

```
<bean id="country"
      class="com.cognizant.springlearn.model.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
</bean>
```

application.properties

```
1 spring.application.name=spring-learn
2 logging.level.com.cognizant=DEBUG
3 server.port=9090
```

Console Output:

```

2026-07-09T14:23:09.701+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.11 with PID 60588 (C:\Users\shiva\ Dow
2026-07-09T14:23:09.703+05:30 DEBUG 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Running with Spring Boot v3.5.16, Spring v6.2.19
2026-07-09T14:23:09.704+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: 'default'
2026-07-09T14:23:09.803+05:30 INFO 60588 [[spring-learn] restartedMain e.DevToolsPropertyDefaultsPostProcessor : DevTools property defaults active! Set 'spring.devtools.add-properties' to 'false' to
2026-07-09T14:23:09.803+05:30 INFO 60588 [[spring-learn] restartedMain e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property
2026-07-09T14:23:09.893+05:30 INFO 60588 [[spring-learn] restartedMain o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 9090 (http)
2026-07-09T14:23:11.719+05:30 INFO 60588 [[spring-learn] restartedMain o.apache.catalina.core.StandardService : Starting service 'Tomcat'
2026-07-09T14:23:11.719+05:30 INFO 60588 [[spring-learn] restartedMain o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-07-09T14:23:11.781+05:30 INFO 60588 [[spring-learn] restartedMain o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-09T14:23:11.783+05:30 INFO 60588 [[spring-learn] restartedMain w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1977 ms
2026-07-09T14:23:11.811+05:30 INFO 60588 [[spring-learn] restartedMain o.s.b.w.a.OptionalWebLoadServer : LiveReload server running on port 35729
2026-07-09T14:23:12.475+05:30 INFO 60588 [[spring-learn] restartedMain o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 9090 (http) with context path '/'
2026-07-09T14:23:12.493+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 3.678 seconds (process running for 4.641)
2026-07-09T14:23:12.503+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Inside main
2026-07-09T14:23:12.503+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Start
2026-07-09T14:23:12.655+05:30 DEBUG 60588 [[spring-learn] restartedMain c.cognizant.springlearn.model.Country : Inside Country Constructor.
2026-07-09T14:23:12.657+05:30 DEBUG 60588 [[spring-learn] restartedMain c.cognizant.springlearn.model.Country : Inside setCode()
2026-07-09T14:23:12.657+05:30 DEBUG 60588 [[spring-learn] restartedMain c.cognizant.springlearn.model.Country : Inside setName()
2026-07-09T14:23:12.658+05:30 DEBUG 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : Country : Country [code=IN, name=India]
2026-07-09T14:23:12.662+05:30 INFO 60588 [[spring-learn] restartedMain c.c.springlearn.SpringLearnApplication : End

```

Hello World RESTful Web Service

Write a REST service in the spring learn application created earlier, that returns the text "Hello World!!" using Spring Web Framework. Refer details below:

Method: GET

URL: /hello

Controller: com.cognizant.spring-learn.controller.HelloController

Method Signature: public String sayHello()

Method Implementation: return hard coded string "Hello World!!"

Sample Request: http://localhost:8083/hello

Sample Response: Hello World!!

IMPORTANT NOTE: Don't forget to include start and end log in the sayHello() method.

Try the URL http://localhost:8083/hello in both chrome browser and postman.

SME to explain the following aspects:

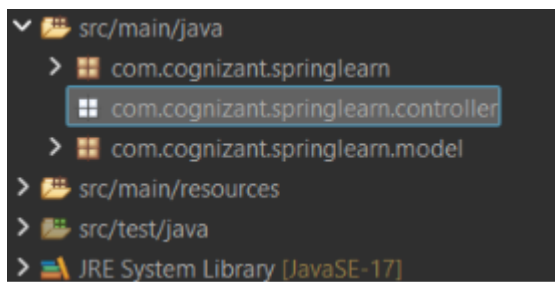
- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

Solution:

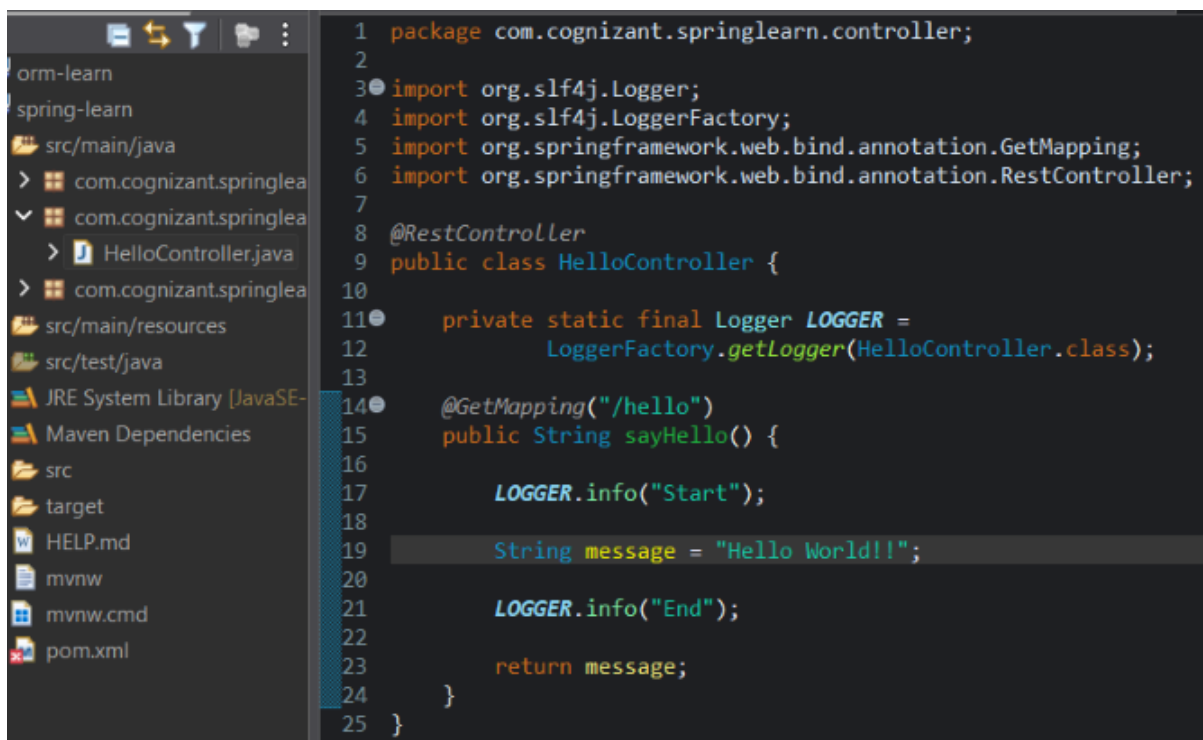
spring-boot-starter-web dependency in pom.xml

```
33 ●      <dependency>
34          <groupId>org.springframework.boot</groupId>
35          <artifactId>spring-boot-starter-web</artifactId>
36      </dependency>
37
```

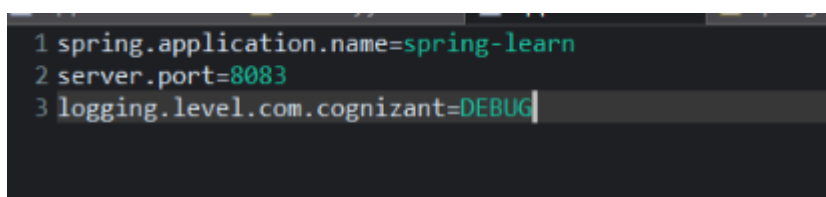
newly created controller



HelloController.java code



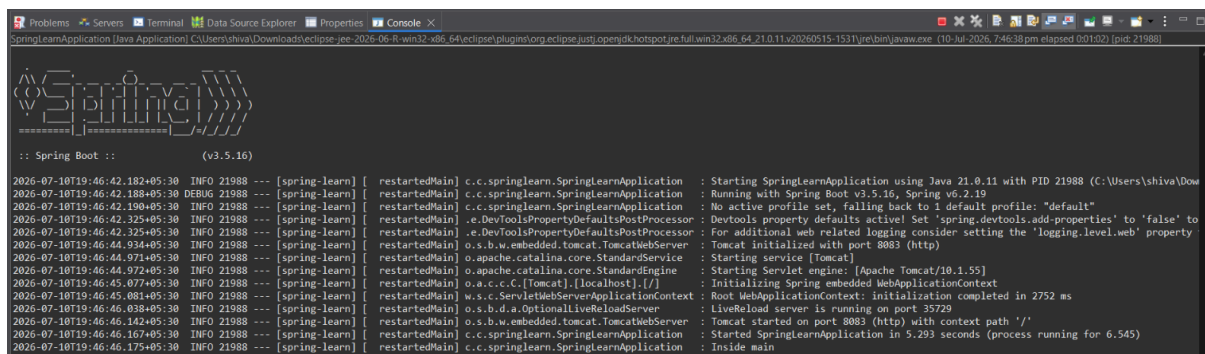
application.properties



SpringLearnApplication.java

```
1 package com.cognizant.springlearn;
2
3 import org.slf4j.Logger;
10
11 @SpringBootApplication
12 public class SpringLearnApplication {
13
14     private static final Logger LOGGER =
15         LoggerFactory.getLogger(SpringLearnApplication.class);
16
17     public static void main(String[] args) {
18
19         SpringApplication.run(SpringLearnApplication.class, args);
20
21         LOGGER.info("Inside main");
22
23     }
24     private static void displayCountry() {
25         LOGGER.info("Start");
26         ClassPathXmlApplicationContext context =
27             new ClassPathXmlApplicationContext("country.xml");
28         Country country =
29             context.getBean("country", Country.class);
30         LOGGER.debug("Country : {}", country);
31
32         context.close();
33
34         LOGGER.info("End");
35     }
36 }
```

OUTPUT:



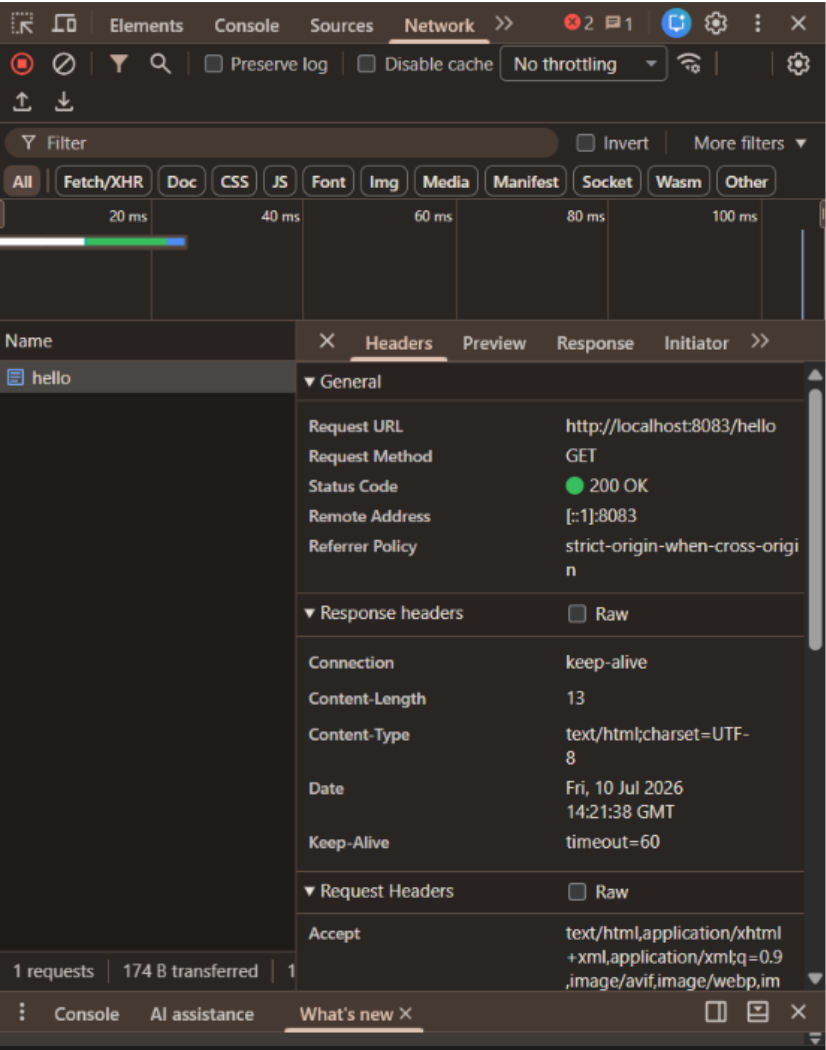
The screenshot shows the Eclipse IDE interface with the console window open. The console displays the Spring Boot application's startup logs. At the top, there is a stylized ASCII art logo for 'Spring Boot'. Below the logo, the text 'Spring Boot :: (v3.5.16)' is shown. The main part of the console contains a series of log entries from the 'spring-learn' application. These logs indicate that the application is starting, running with Spring Boot v3.5.16 and Spring v6.2.19. It shows the initialization of the Tomcat web server on port 8083 (http) and the starting of the service. The logs also mention the initialization of the Spring embedded WebApplicationContext and the LiveReload server. Finally, the logs show that the application has started successfully in 5.293 seconds and is now inside the main method.

```
SpringLearnApplication [Java Application] C:\Users\shiva\Downloads\eclipse-je-2026-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\re\bin\java.exe (10-Jul-2026, 7:46:38 pm elapsed 0:01:02) [pid: 21988]

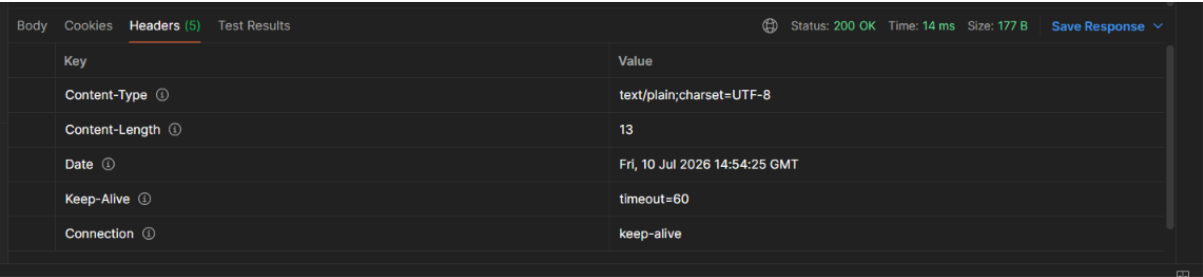
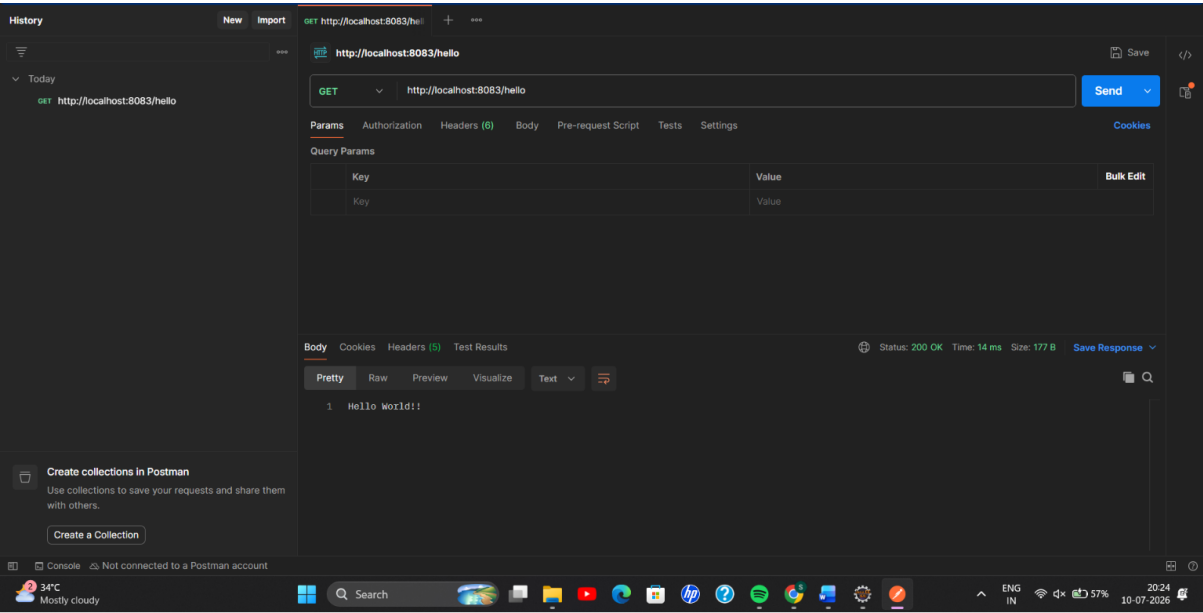
:: Spring Boot :: (v3.5.16)

2026-07-10T19:46:42.182+05:30 INFO 21988 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 21.0.11 with PID 21988 (C:\Users\shiva\Dow
2026-07-10T19:46:42.188+05:30 DEBUG 21988 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Running with Spring Boot v3.5.16, Spring v6.2.19
2026-07-10T19:46:42.190+05:30 INFO 21988 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-10T19:46:42.325+05:30 INFO 21988 --- [spring-learn] [ restartedMain] e.DevToolsPropertyDefaultsPostProcessor : DevTools property defaults active! Set 'spring.devtools.add-properties' to 'false' to
2026-07-10T19:46:42.325+05:30 INFO 21988 --- [spring-learn] [ restartedMain] e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property
2026-07-10T19:46:44.934+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8083 (http)
2026-07-10T19:46:44.971+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Starting service [Tomcat]
2026-07-10T19:46:44.972+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-07-10T19:46:45.877+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-10T19:46:45.881+05:30 INFO 21988 --- [spring-learn] [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2752 ms
2026-07-10T19:46:46.038+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-07-10T19:46:46.142+05:30 INFO 21988 --- [spring-learn] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8083 (http) with context path '/'
2026-07-10T19:46:46.167+05:30 INFO 21988 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 5.293 seconds (process running for 6.545)
2026-07-10T19:46:46.175+05:30 INFO 21988 --- [spring-learn] [ restartedMain] c.c.springlearn.SpringLearnApplication : Inside main
```

Chrome Browser:



Postman:



REST - Country Web Service

Write a REST service that returns India country details in the earlier created spring learn application.

URL: /country

Controller: com.cognizant.spring-learn.controller.CountryController

Method Annotation: @RequestMapping

Method Name: getCountryIndia()

Method Implementation: Load India bean from spring xml configuration and return

Sample Request: http://localhost:8083/country

Sample Response:

```
{  
  "code": "IN",  
  "name": "India"  
}
```

SME to explain the following aspects:

- What happens in the controller method?
- How the bean is converted into JSON response?
- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

```

  ____  __
 / ___/  / /_  __
/ /   / __/ / / /
/ /___/ /_/ /_/ /
/_____/_/

:: Spring Boot ::

(v4.1.0)

2026-07-11T21:27:21.124+05:30 INFO 21352 --- [main] o.e.c.CountrywebserviceApplication : Starting CountrywebserviceApplication using Java 17.0.10 with PID 21
2026-07-11T21:27:21.127+05:30 INFO 21352 --- [main] o.e.c.CountrywebserviceApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-11T21:27:21.783+05:30 INFO 21352 --- [main] o.s.boot.TomcatTomcatWebServer : Tomcat initialized with port 8083 (http)
2026-07-11T21:27:21.794+05:30 INFO 21352 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-11T21:27:21.795+05:30 INFO 21352 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-11T21:27:21.841+05:30 INFO 21352 --- [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: Initialization completed in 669 ms
2026-07-11T21:27:22.171+05:30 INFO 21352 --- [main] o.s.boot.TomcatTomcatWebServer : Tomcat started on port 8083 (http) with context path '/'
2026-07-11T21:27:22.175+05:30 INFO 21352 --- [main] o.e.c.CountrywebserviceApplication : Started CountrywebserviceApplication in 1.466 seconds (process running)
2026-07-11T21:27:39.143+05:30 INFO 21352 --- [nio-8083-exec-1] o.e.c.c.g.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-11T21:27:39.144+05:30 INFO 21352 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-11T21:27:39.146+05:30 INFO 21352 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms

```



Controller: com.cognizant.spring-learn.controller.CountryController

Method Name: `getCountry(String code)`

Service Method: com.cognizant.spring-

```
learn.service.CountryService.getCountry(String code)
```

Service Method Implementation:

- Get the country code using @PathVariable
- Get country list from country.xml
- Iterate through the country list
- Make a case insensitive matching of country code and return the country.
- Lambda expression can also be used instead of iterating the country list

Sample Request: http://localhost:8083/country/in

Sample Response:

```
{
  "code": "IN",
  "name": "India"
}
```

OUTPUT:

```

  ____  _
 / ___|| | | |
| |___| |_| |
|___ \\__|_|_|_|

:: Spring Boot ::      (v4.1.0)

2026-07-09T17:01:54.906+05:30 INFO 21220 --- [main] o.e.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 17.0.10 with PID 21220 (C
2026-07-09T17:01:54.908+05:30 INFO 21220 --- [main] o.e.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-09T17:01:55.502+05:30 INFO 21220 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8083 (http)
2026-07-09T17:01:55.513+05:30 INFO 21220 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-09T17:01:55.513+05:30 INFO 21220 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-09T17:01:55.550+05:30 INFO 21220 --- [main] o.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 603 ms
2026-07-09T17:01:55.855+05:30 INFO 21220 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8083 (http) with context path '/'
2026-07-09T17:01:55.859+05:30 INFO 21220 --- [main] o.e.springlearn.SpringLearnApplication : Started SpringLearnApplication in 1.358 seconds (process running for
2026-07-09T17:02:39.635+05:30 INFO 21220 --- [nio-8083-exec-1] o.e.c.s.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-09T17:02:39.636+05:30 INFO 21220 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-09T17:02:39.638+05:30 INFO 21220 --- [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms

```

←
→
↺
ⓘ localhost:8083/countries/in

Pretty-print ☐

```
{ "code": "IN", "name": "India" }
```


← → ↻ localhost:8090/authenticate

Pretty-print ☐

```
{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI1c2VyIiwiaWF0IjoxNzg5NTk0MDA5LCJleHAiOiJlE3ODM1OTU0MD19.mBpNLRqXgA7XcHhF9ET0z8_c6AaDS25aB62jT_vgL7g"}
```